

# Programmation Javascript côté serveur avec Node.Js

## Partie 1 : Weather App

### 1. Utiliser l'API Web « Geolocation » côté navigateur

L'API Web « Geolocation » permet de connaître la géolocalisation d'un utilisateur.

Elle est accessible via un appel à la propriété « `navigator.geolocation` » du navigateur. Pour des raisons de confidentialité, le navigateur va demander à l'utilisateur l'autorisation d'accéder à ses données de localisation.

Si l'utilisateur accepte de partager les données de localisation, le navigateur utilisera la meilleure fonctionnalité disponible sur l'appareil pour accéder à ces informations (GPS, données de connexion à Internet...).

Vous trouverez plus de détails à l'adresse suivante : [https://developer.mozilla.org/en-US/docs/Web/API/Geolocation\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API).

#### TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Créez un nouveau projet Node.js nommé « WeatherApp ».
- Installez « Express » puis créez un dossier « public ».
- Ajoutez ensuite un fichier « index.html » dans le dossier « public ».
- Modifier le fichier « index.js » pour exécuter « Express » sur le port 3000 et afficher le fichier « Index.html ».

Vérifiez que l'API « Geolocation » est disponible dans votre navigateur en modifiant le fichier « index.html » :

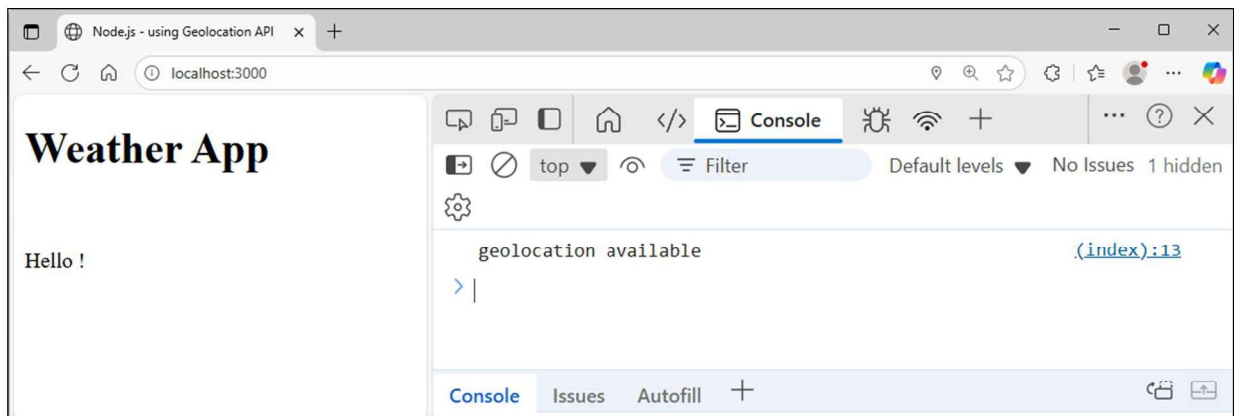
```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Node.js - using Geolocation API</title>
  </head>
  <body>
    <script>
      if ("geolocation" in navigator) {
        /* geolocation is available */
        console.log('geolocation available');
      } else {
        /* geolocation IS NOT available */
        console.log('geolocation not available');
      }
    </script>
    <h1>Weather App</h1>
    <br>
    <p> Hello ! </p>
```

```
</body>
</html>
```

Testez le résultat dans votre navigateur. Commencez par accepter de partager votre localisation : la console en utilisant le mode développeur du navigateur :



Visualisez le résultat dans la console en utilisant le mode développeur du navigateur :

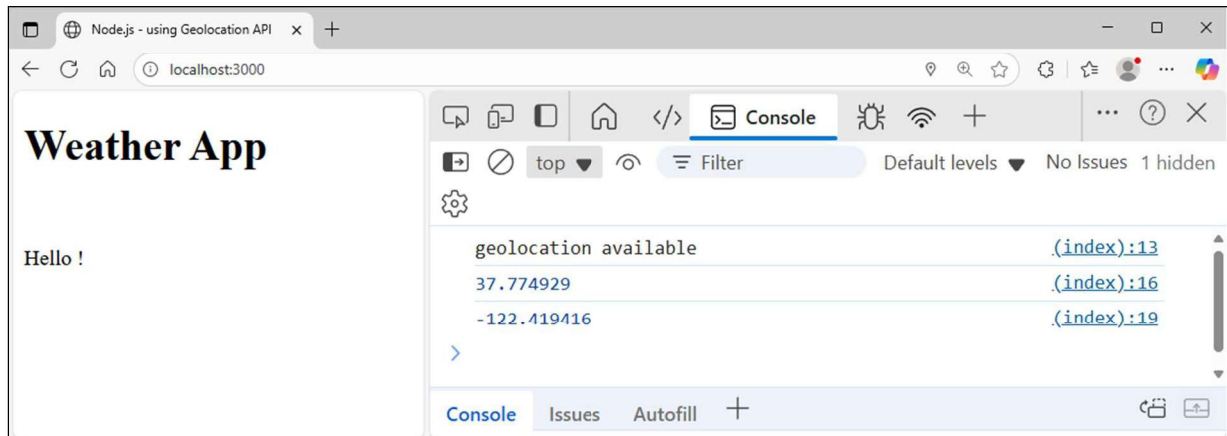


Récupérez ensuite les coordonnées GPS en utilisant la fonction « `GetCurrentPosition()` » dans le code du fichier « `index.html` » :

```
<script>
  if ("geolocation" in navigator) {
    /* geolocation is available */
    console.log('geolocation available');
    navigator.geolocation.getCurrentPosition((position) => {
      // => Quand c'est prêt, alors...
      console.log(position.coords.latitude);
      console.log(position.coords.longitude);
    });
  } else {
    /* geolocation IS NOT available */
    console.log('geolocation not available');
  }
</script>
```

**Remarque :** nous utilisons ici une « Arrow function » (on exécute les instructions prévues seulement une fois que les coordonnées sont récupérées).

Observez le résultat dans la console :



Il nous reste à présent à afficher les coordonnées dans notre page Web.

Commencez par modifier la page « index.html » pour ajouter les deux éléments à afficher :

```
...
<h1>Weather App</h1>
<br>
<p>
  <br>
  Latitude : <span id="latitude"></span> &deg;
  /* &deg ; permet d'afficher le caractère « ° » */
  <br>
  Longitude : <span id="longitude"></span> &deg;
</p>
</body>
```

Modifiez ensuite le script pour stocker la latitude et la longitude dans des variable dès qu'elles sont récupérées et valoriser les conteneurs `<span id="latitude"></span>` et `<span id="longitude"></span>` :

```
<script>
  if ("geolocation" in navigator) {
    /* geolocation is available */
    console.log('geolocation available');

    navigator.geolocation.getCurrentPosition((position) => {
      // => Quand c'est prêt, alors...
      console.log(position.coords.latitude);

      const lat = position.coords.latitude;
      document.getElementById('latitude').textContent = lat;
    });
  }
}
```

```

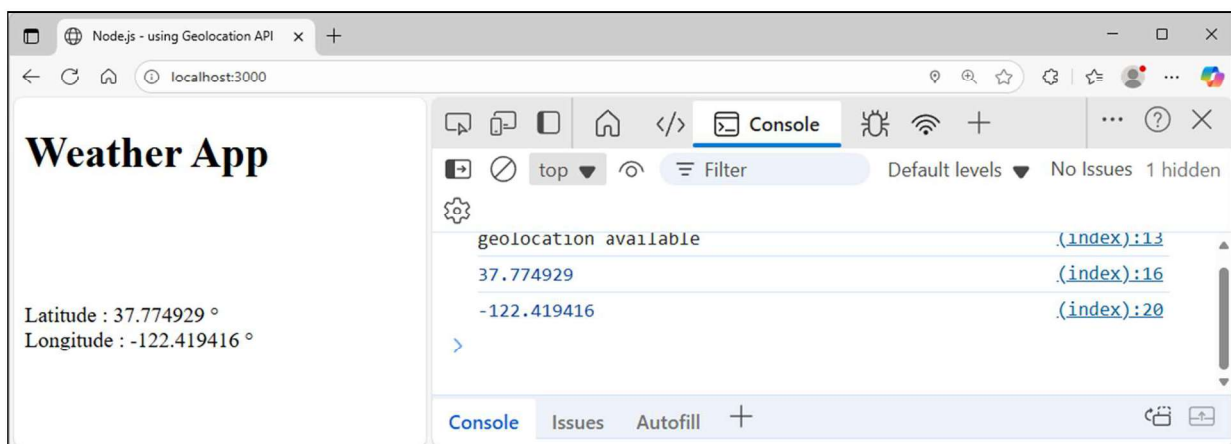
        console.log(position.coords.longitude);

        const long = position.coords.longitude;
        document.getElementById('longitude').textContent = long;
    });
} else {
    /* geolocation IS NOT available */
    console.log('geolocation not available');
}
</script>

```

**Remarque :** l'instruction « `document.getElementById()` » renvoie l'objet dont la propriété « `id` » correspond à la chaîne de caractères spécifiée en paramètre. On valorise ici la propriété « `textContent` » de l'objet dont l'« `id` » est « `longitude` ».

Voici le résultat final :



## 2. POST Request avec fetch()

Ici, nous allons utiliser la méthode « `POST` » pour que notre client (ici le fichier « `index.html` » interprété par un navigateur) puisse envoyer des données au format JSON à notre serveur « `Node.js` ».

La première étape consiste à créer sur notre serveur une nouvelle route dérivée de la méthode http « `POST` ». Nous allons attacher cette route à une instance du Framework de création d'application web que nous avons déjà utilisé : « `Express` ».

**Remarque :** le « `routing` » fait référence à la façon dont les points de terminaison (URI) d'une application répondent aux demandes des clients.

N'hésitez pas à consulter la documentation d'Express à l'adresse suivante :

<https://expressjs.com/en/guide/routing.html> .

Ouvrez votre projet « WeatherApp » puis modifiez le code du fichier « index.js » :

```
const express = require('express');

const app = express();
app.listen(3000, () => console.log('listening at 3000'));
app.use(express.static('public')); //répertoire accessible à tous qui contient mes pages web
app.use(express.json({limit: '1mb'})); //utilisation de la méthode express.json() en limitant
la taille à 1 MegaBytes

//POST method route
app.post('/api', (request, response) => { // => Quand c'est prêt, alors...
  console.log('I got a request');
  console.log(request.body);
  const data = request.body;
  response.end(); //on renvoie une réponse au message (obligatoire)
});
```

**Remarque :** n'oubliez pas d'ajouter le code qui permet d'ajouter la méthode « express.json() » à notre programme. Cette méthode est indispensable pour décoder et afficher le résultat envoyé au format JSON.

Modifiez ensuite le code du fichier « index.html » pour envoyer nos coordonnées en utilisant la route « /api » :

```
<script>
  if ("geolocation" in navigator) {
    /* geolocation is available */
    console.log('geolocation available');

    navigator.geolocation.getCurrentPosition((position) => {
      // => Quand c'est prêt, alors...
      console.log(position.coords.latitude);

      const lat = position.coords.latitude;
      document.getElementById('latitude').textContent = lat;
      console.log(position.coords.longitude);

      const long = position.coords.longitude;
      document.getElementById('longitude').textContent = long;

      const data = {lat,long};
      const options = {
        method: 'POST',
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(data)
      }
    });
  }
}
```

```

    });
    fetch('/api',options);
  });

  } else {
    /* geolocation IS NOT available */
    console.log('geolocation not available');
  }
</script>

```

- L'instruction `fetch('/api',options);` permet d'utiliser la route « /api » pour envoyer notre requête.
- La constante « `options` » qui est passée en paramètre permet d'indiquer au serveur quelle est la méthode utilisée (ici, la méthode « POST »), le type de données envoyées (ici, des données au format JSON) et d'envoyer les données dans le paramètre « `body` » de la requête.

Visitez la page [https://developer.mozilla.org/fr/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/fr/docs/Web/API/Fetch_API/Using_Fetch) pour plus d'informations

**Remarque :** à chaque changement dans votre code, vous devez redémarrer votre serveur « Node.js » pour voir le résultat. Si vous souhaitez que les modifications soient automatiquement prises en compte, vous pouvez utiliser le module « nodemonitor », qui va redémarrer votre serveur après chaque enregistrement.

Lancez la commande « `npm install -g nodemon` » puis démarrez votre serveur avec la commande « `nodemon index.js` ».

**Testez votre programme** en actualisant la page `index.html` dans votre navigateur et observez le résultat côté serveur. Le serveur reçoit bien nos coordonnées :

```

PS C:\NodeJs\WeatherApp> node index.js
listening at 3000
I got a request
{ lat: 51.507351, long: -0.127758 }
|

```

Améliorons maintenant notre programme en envoyant une réponse à notre client (notre page « `index.html` »).

Commencez par ajouter le code suivant dans le fichier « `index.js` » :

```

//POST method route
app.post('/api', (request, response) => { // => Quand c'est prêt, alors...
  console.log('I got a request');
  console.log(request.body);
  const data = request.body;

  //Response
  response.json({

```

```

        status: 'success',
        latitude: data.lat,
        longitude: data.long
    });
});

```

Modifiez ensuite le code du fichier « index.html » :

```

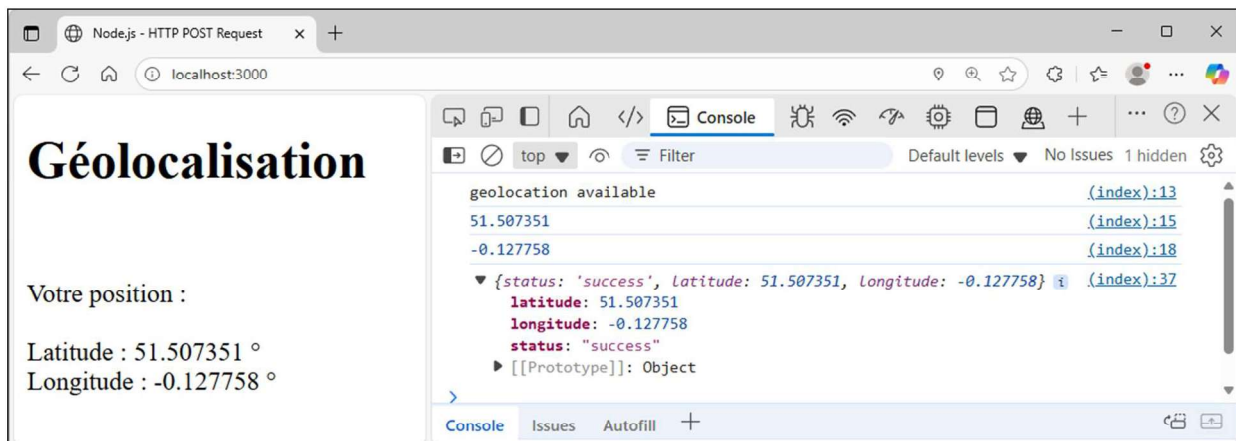
const data = {lat,long};
const options = {
  method: 'POST',
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(data)
};

//Requête avec Fetch() en utilisant des fonctions asynchrones
const response = await fetch('/api',options);
const dataResponse = await response.json();
console.log(dataResponse);

```

Ici, on utilise une fonction asynchrone pour attendre la réponse et une fonction asynchrone pour attendre la mise en forme des données au format JSON.

Observez le résultat en actualisant la page « index.html » dans votre navigateur :



La réponse est affichée dans la console au format JSON. Parfait !

### 3. Stocker les données dans un tableau

Notre objectif est ici d'ajouter sur la page « index.html » un bouton qui déclenche l'envoi des coordonnées géographiques au serveur Node.js, puis d'ajouter ces données à un tableau de coordonnées.

Commencez par modifier le fichier « index.js » pour créer un tableau et ajouter les données lors de l'exécution de chaque requête « POST » :

```
const dataArray = []; //Tableau pour stocker les informations (une ligne par request)

//POST method route
app.post('/api', (request, response) => { // => Quand c'est prêt, alors...
    console.log('I got a request');
    console.log(request.body);
    const data = request.body;
    dataArray.push(data); //on ajoute les données au tableau
    console.log(dataArray);

    //on renvoie une réponse au message (obligatoire)
    response.json({
        status: 'success',
        data: dataArray
    });
});
```

Il nous reste maintenant à ajouter un bouton et à créer un événement qui envoie la requête au serveur :

```
<body>
  <h1>Géolocalisation</h1>
  <br>
  <p>
    Votre position :
    <br>
    <br>
    Latitude : <span id="latitude"></span> °
    <br>
    Longitude : <span id="longitude"></span> °
    <br>
    <br>
    <button id="submit">Envoyer</button> <!-- Bouton d'envoi des informations -->
  </p>
  <script>
    let lat;
    let long;

    const button = document.getElementById('submit');

    button.addEventListener('click', async event => {
      const data = {lat,long};
      const options = {
        method: 'POST',
```

```

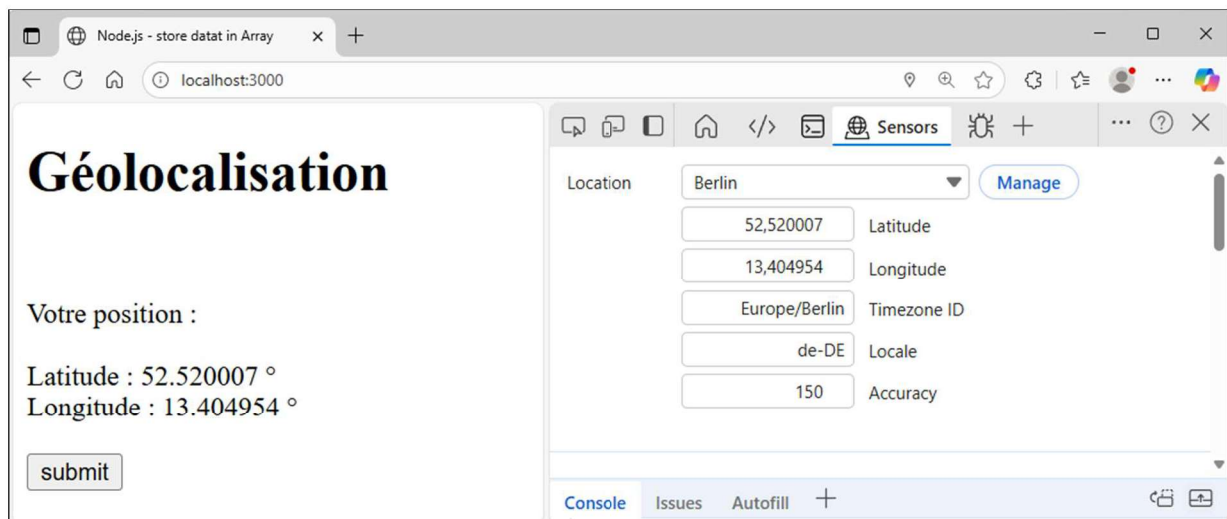
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(data)
    });

    const response = await fetch('/api', options);
    const dataResponse = await response.json();
    console.log(dataResponse);
});

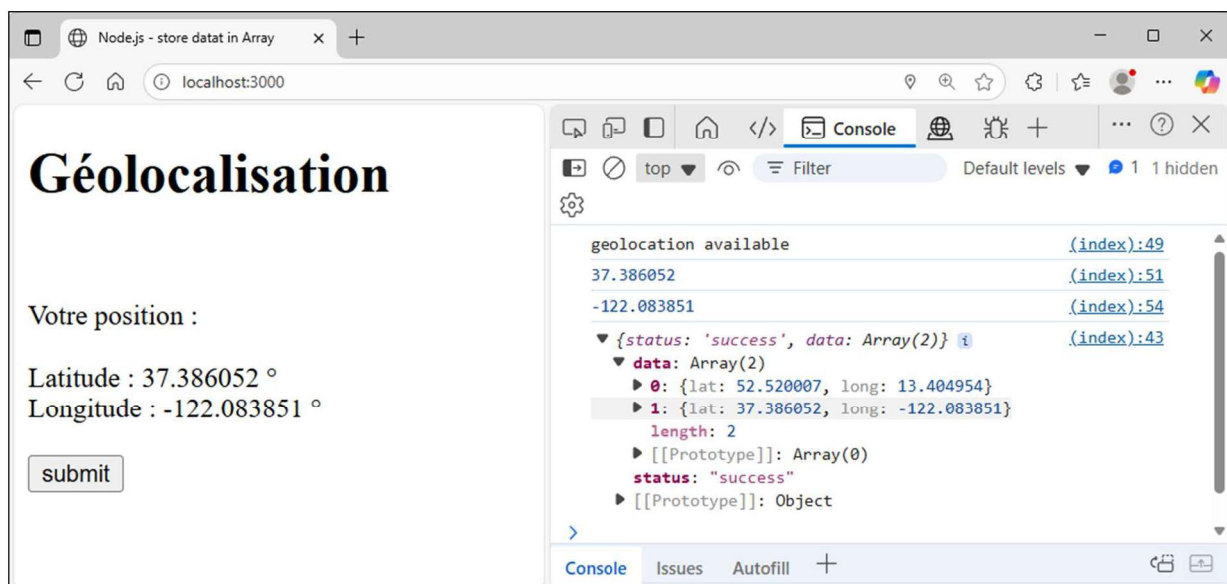
```

Pour tester, le résultat, nous allons utiliser le mode développeur et modifier notre géolocalisation avant chaque envoi vers le serveur.

Entrez dans le mode développeur en utilisant la touche « F12 » puis cliquez sur l'onglet « Sensors » et modifiez la localisation avant chaque envoi (réactualisez la page avant de cliquer sur le bouton « submit ») :



Observez le résultat dans la console du navigateur :



Et le résultat côté serveur :

```
[nodemon] starting `node index.js`  
listening at 3000  
I got a request  
{ lat: 52.520007, long: 13.404954 }  
[ { lat: 52.520007, long: 13.404954 } ]  
I got a request  
{ lat: 37.386052, long: -122.083851 }  
[  
  { lat: 52.520007, long: 13.404954 },  
  { lat: 37.386052, long: -122.083851 }  
]
```

Nous avons réussi à stocker les données sur notre serveur Node.js !

### TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Importez la bibliothèque « Bootstrap » dans votre page « index.html »
- Modifiez le « front-end » avec « Bootstrap » pour rendre votre site « responsive ».

## 4. Enregistrer les données au format JSON dans un fichier texte

Il s'agit ici de s'assurer de la persistance des données, en enregistrant les coordonnées dans un fichier nommé « database.txt ».

Nous allons utiliser le module « File system ». Comme ce module est inclus par défaut dans « Node.js », il suffit de l'importer dans notre fichier « index.js » avec l'instruction suivante : `const fs = require("fs");`

Commencez par ajouter une nouvelle fonction pour enregistrer les données dans le fichier « index.js » :

```
function saveToDatabase(data) {  
  const dataString = JSON.stringify(data);  
  fs.appendFileSync("database.txt", dataString + "\n");  
}
```

### Remarques :

- L'instruction « `JSON.stringify()` » permet de convertir un type de données Javascript en une chaîne de caractères au format JSON.
- L'instruction « `appendFileSync("database.txt", dataString + "\n")` » ajoute de manière synchrone les données dans le fichier « database.txt ». Si le fichier n'existe pas, il est créé automatiquement. On ajoute un saut de ligne avec le caractère « `\n` ».

Ajoutez ensuite une fonction « `getDatabase()` » qui va nous permettre de récupérer les données enregistrées dans le fichier « database.txt » puis de les stocker dans un tableau :

```
function getDatabase() {  
  const rawData = fs.readFileSync("database.txt").toString().trim();  
  const lines = rawData.split("\n");  
  const allData = [];
```

```

    for (const line of lines) {
        const data = JSON.parse(line);
        allData.push(data);
    }
    return allData;
}

```

Il nous reste à modifier notre route « /api » pour enregistrer dans notre fichier texte chaque envoi de coordonnées.

```

app.post('/api', (request, response) => { // => Quand c'est prêt, alors...
    console.log('I got a request');
    console.log(request.body);
    const data = request.body;
    saveToDatabase(data); //on ajoute les données envoyées dans le fichier de sauvegarde
    const allData = getDatabase(); //on récupère la totalité de données sauvegardées
    response.json(allData); //on envoie la totalité des données comme réponse
    console.log(allData);
});

```

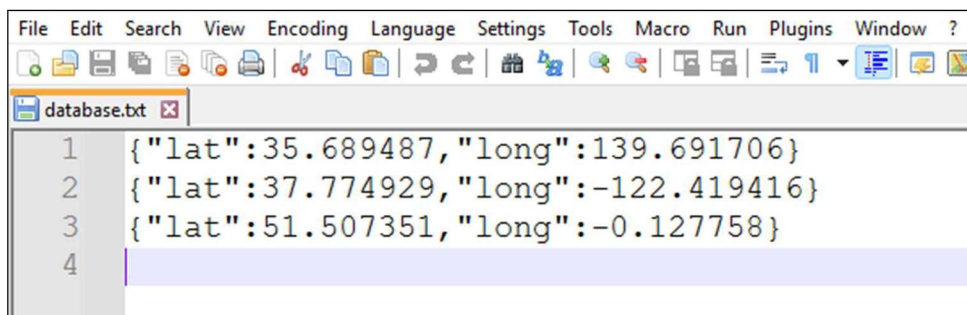
La réponse envoyée au serveur permet de vérifier que les données sont bien enregistrées :

```

[nodemon] starting `node index.js`
listening at 3000
I got a request
{ lat: 35.689487, long: 139.691706 }
[ { lat: 35.689487, long: 139.691706 } ]
I got a request
{ lat: 37.774929, long: -122.419416 }
[
  { lat: 35.689487, long: 139.691706 },
  { lat: 37.774929, long: -122.419416 }
]
I got a request
{ lat: 51.507351, long: -0.127758 }
[
  { lat: 35.689487, long: 139.691706 },
  { lat: 37.774929, long: -122.419416 },
  { lat: 51.507351, long: -0.127758 }
]

```

Vérifiez que les données sont bien enregistrées dans votre fichier « database.txt » :



```

1 {"lat":35.689487,"long":139.691706}
2 {"lat":37.774929,"long":-122.419416}
3 {"lat":51.507351,"long":-0.127758}
4

```

**Remarque :** si nécessaire, vous pouvez effacer le contenu du fichier en ajoutant une nouvelle fonction « clearDatabase() ».

Il suffira par la suite de l'appeler pour remettre à zéro vos données.

```
function clearDatabase() {  
  fs.writeFileSync("database.txt", "");  
}
```

## 5. Enregistrer les données dans une base de données SQLite

Nous allons maintenant utiliser une base de données SQLite pour enregistrer nos coordonnées ainsi que d'autres informations.

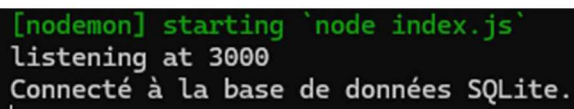
Commencez par créer un nouveau projet « WeatherAppSQLite » qui reprend le code du projet « WeatherApp ».

Installez ensuite le package « SQLite3 » avec la commande « `npm install sqlite3` ».

Ouvrez ensuite le fichier « index.js » et modifiez le code pour importer « SQLite3 » et ouvrir une nouvelle base de données :

```
const sqlite3 = require('sqlite3').verbose(); //on importe SQLite3  
  
const express = require('express');  
const app = express();  
app.listen(3000, () => console.log('listening at 3000'));  
app.use(express.static('public')); //repertoire accessible à tous qui contient mes pages web  
app.use(express.json({limit: '1mb'})); //utilisation de la fonction express.json() en  
limitant la taille à 1 MegaBytes  
  
// Ouvrir une base de données SQLite  
const db = new sqlite3.Database('./weatherApp.db', (err) => {  
  if (err) {  
    console.log(err.message);  
  } else {  
    console.log('Connecté à la base de données SQLite.');  }  
});
```

Testez la connexion :



```
[nodemon] starting `node index.js`  
listening at 3000  
Connecté à la base de données SQLite.
```

Ajoutez le code suivant juste après l'ouverture de la base de données pour créer une nouvelle table dans notre base « weatherApp.db » :

```
db.serialize(function() {  
  let sql = 'CREATE TABLE IF NOT EXISTS geoloc(id INTEGER PRIMARY KEY, latitude TEXT NOT NULL,  
longitude TEXT NOT NULL, timestamp REAL NOT NULL)';
```

```

db.run(sql, (err) => {
  if (err) {
    console.log(err.message);
  } else {
    console.log('Création de la table geoloc');
  }
});
});

```

Si tout se passe bien, une nouvelle table « geoloc » est ajoutée à la base de données :

```

[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'
listening at 3000
Connecté à la base de données SQLite.
Création de la table geoloc

```

### Remarques :

- La table n'est créée que lors de la première exécution (« CREATE TABLE IF NOT EXISTS ... »).
- Quatre champs sont créés : id (clé primaire), latitude, longitude et timestamp (date et heure de création de l'enregistrement).

Ajoutez maintenant une fonction pour insérer des données :

```

function Insérer(lat, long, timestamp){

  db.serialize(function() {
    const stmt = db.prepare('INSERT INTO geoloc(latitude, longitude, timestamp)
VALUES (?, ?, ?)');
    stmt.run(lat, long, timestamp);

    stmt.finalize((err) => {
      if (err) {
        console.log(err.message);
      } else {
        console.log('Insert réussi.');
```

Puis une fonction pour afficher toutes les données dans la console :

```

function Afficher(){
  db.serialize(function() {
    db.each("SELECT id, latitude, longitude, timestamp FROM geoloc", (err, row) => {
      console.log(row.id + " : " + row.latitude + " " + row.longitude + " " + row.timestamp);
    });
  });
}

```

Pour enregistrer nos données et afficher le résultat dans la console du navigateur, nous allons créer une fonction asynchrone « `databaseInsert()` » (insérer prend du temps et il faut attendre la fin de l'opération pour afficher l'enregistrement dans la base) :

```
async function databaseInsert(data){
    let resultInsert = await Inserir(data.lat,data.long,data.timestamp);
    let resultAfficher = await Afficher();
};
```

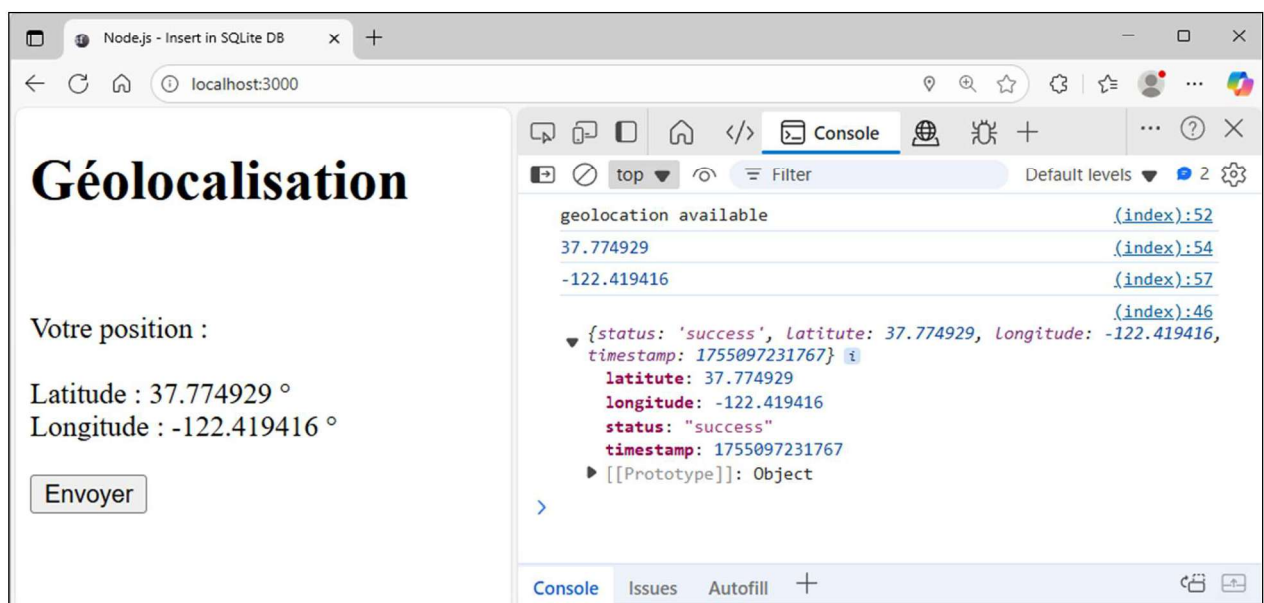
Il ne reste plus qu'à modifier notre route pour envoyer les données à la fonction « `databaseInsert()` » :

```
//POST method route
app.post('/api', (request, response) => { // => Quand c'est prêt, alors...
    console.log('I got a request');
    console.log(request.body);

    const data = request.body;
    const timestamp = Date.now();
    data.timestamp = timestamp;
    databaseInsert(data);
    console.log(data);

    response.json({
        status: 'success',
        latitude: data.lat,
        longitude: data.long,
        timestamp: data.timestamp
    });
});
```

Ouvrez votre navigateur et essayez d'envoyer plusieurs fois votre position :



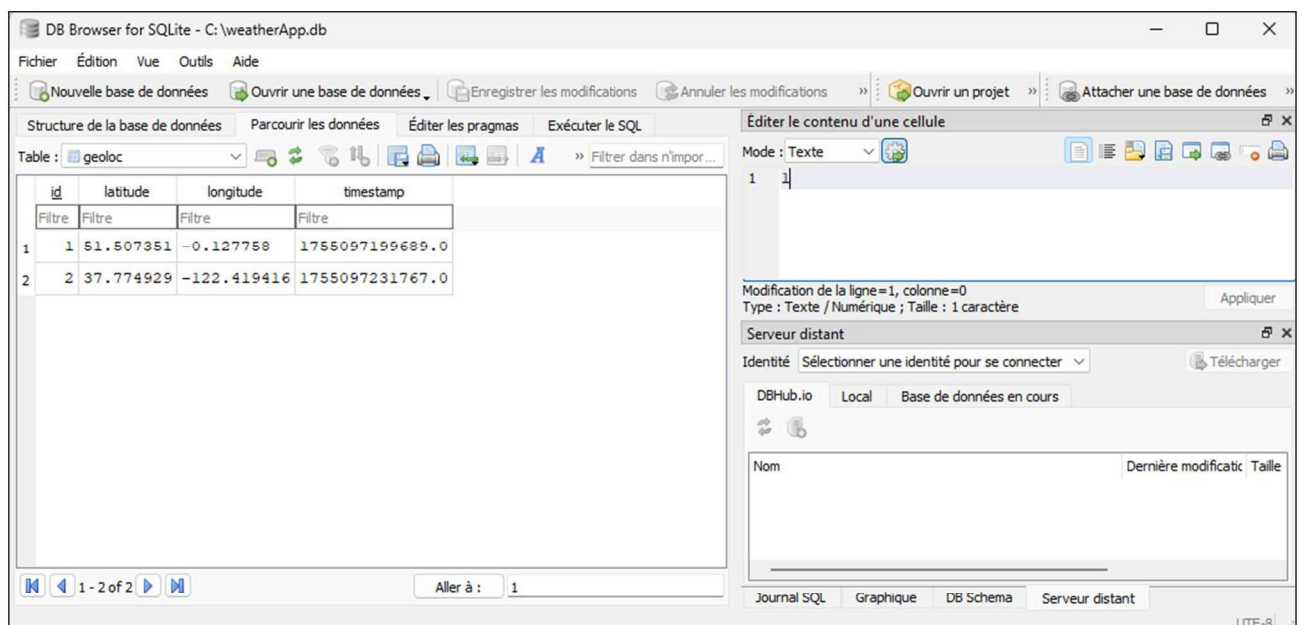
Observez les logs dans la console du navigateur : seules les dernières données enregistrées sont affichées.

Côté serveur, toutes les données présentes dans la base sont affichées (fonction Afficher()) dans « index.js ») :

```
[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'
listening at 3000
Connecté à la base de données SQLite.
Création de la table geoloc
I got a request
{ lat: 51.507351, long: -0.127758 }
{ lat: 51.507351, long: -0.127758, timestamp: 1755097199689 }
Insert réussi.
1: 51.507351 -0.127758 1755097199689.0
I got a request
{ lat: 37.774929, long: -122.419416 }
{ lat: 37.774929, long: -122.419416, timestamp: 1755097231767 }
Insert réussi.
1: 51.507351 -0.127758 1755097199689.0
2: 37.774929 -122.419416 1755097231767.0
```

Pour visualiser les enregistrements dans la base de données, téléchargez « DB Browser for SQLite » à l'adresse suivante : <https://sqlitebrowser.org/>

Ouvrez la base « weatherApp.db » dans « DB Browser for SQLite » :



Les données sont bien enregistrées dans la base !

## 6. Exécuter des requêtes dans la base de données SQLite

### TRAVAIL A FAIRE POUR LES ETUDIANTS :

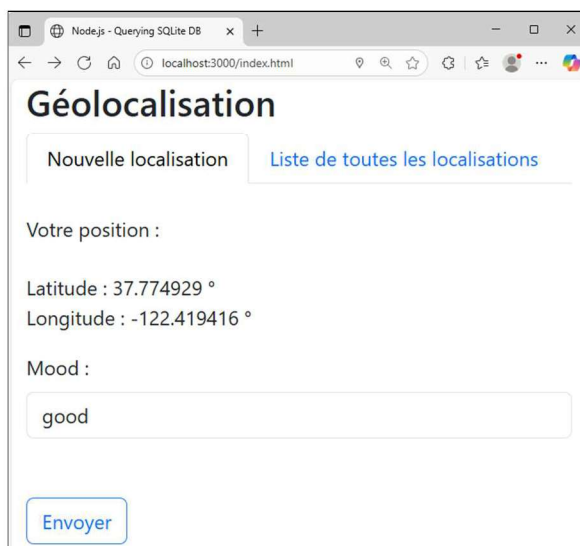
- Ajouter un champ texte « Mood » lors de la création de la table « geoloc » dans la base de données SQLite.
- Ajoutez la possibilité pour l'utilisateur de saisir son humeur (« mood ») sur la page « index.html » et enregistrez sa réponse dans la base lors de l'envoi des données.
- Créer une nouvelle page « all.html » qui affiche l'ensemble des enregistrements de la base de données « weatherApp.db ».

**Remarque :** vous pouvez ajouter des éléments HTML dans du code Javascript. N'hésitez pas à consulter la documentation à l'adresse suivante :

<https://developer.mozilla.org/fr/docs/Web/API/Document/createElement>

- Modifiez les pages « index.html » et « all.html » pour afficher un menu qui permet de passer de la page d'envoi des coordonnées à la page d'affichage de l'ensemble des enregistrements.
- Utilisez « Bootstrap » pour rendre le site « responsive ».
- Exemples :

Page « index.html »



Page « all.html »

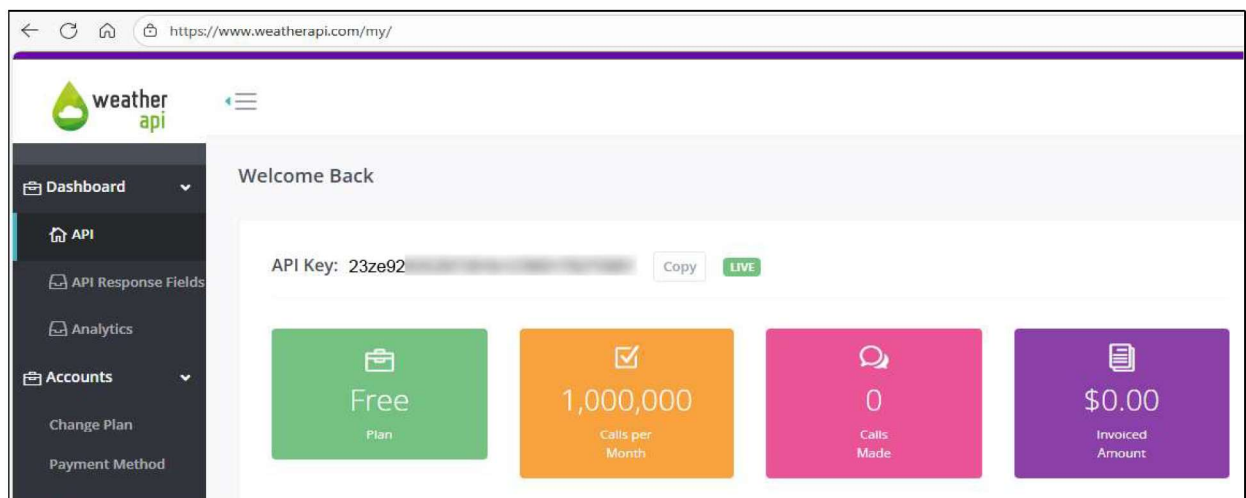


## 7. Utiliser une API pour récupérer des données météorologiques

Ici, nous allons utiliser l'API « Weatherapi » pour récupérer des données géographiques (ville, pays) ainsi que des données météorologiques (le temps qu'il fait, la température, la qualité de l'air et la concentration en particules fines).

Commencez par vous rendre sur le site « <https://www.weatherapi.com/> » et créez un compte (pendant 14 jours vous bénéficiez d'une gratuitement d'une version Pro puis vous passez automatiquement à la version de base, 100% gratuite). Nous n'utiliserons ici que les fonctions de la version 100% gratuite.

Connectez-vous à votre compte et allez dans votre tableau de bord (« Dashboard »). Récupérez votre « API Key ». Cette clé sera nécessaire pour utiliser l'API dans notre application.



Créez un nouveau projet « WeatherAppApi » en recopiant le répertoire de notre projet précédent (l'application « WeatherApp »).

Commencez par modifier le code du fichier « public/index.html » pour afficher tous les nouveaux éléments :

```
<body>
  <div class="container">
    <h1>La météo ici</h1>
    <ul class="nav nav-tabs">
      <li class="nav-item">
        <a class="nav-link active" href="/index.html">Nouvelle localisation</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="/all.html">Liste de toutes les localisations</a>
      </li>
    </ul>
    <br>
    <p>
      Votre position :
    <br>
```

```

<br>
Latitude : <span id="latitude"></span> &deg;
<br>
Longitude : <span id="longitude"></span> &deg;
<br>
<br>
Ville : <span id="city"></span>
<br>
Pays : <span id="country"></span>
<br><br>
<img id="icon" alt="Weather icon">
<br>
<span id="summary"></span>
<br>
Température : <span id="temperature"></span> &deg; C
<br><br>
Niveau de pollution de l'air(index UK DEFRA) : <span id="airQualityIndex"></span>
<br>
Concentration de particules fines (PM2,5) : <span id="pm25"></span>(µg/m3)
<div>
  <label for="moodLabel" class="form-label">Mood :</label>
  <input type="text" class="form-control" id="mood" value="good" minlength="4"
    maxlength="10" size="10" />
</div>
<br><br>
<button class="btn btn-outline-primary" id="submit">Envoyer</button>
</p>
</div>
...

```

Toujours dans le fichier « public\index.html », nous allons commencer par modifier le script principal pour préparer l'envoi des nouvelles données avec la méthode POST :

```

<script>
  let lat;
  let long;
  let city;
  let country;
  let condition ;
  let temperature ;
  let defra;
  let pm25 ;
  const button = document.getElementById('submit');
  button.addEventListener('click', async event => {
    const mood = document.getElementById('mood').value; //On récupère la valeur du champ
    de formulaire "mood"
  });

```

```

const data = {lat,long,mood,city,country,condition,temperature,defra,pm25}; //On
    ajoute les nouvelles données à envoyer
const options = {
method: 'POST',
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(data)
};
const response = await fetch('/api',options);
const dataResponse = await response.json();
console.log(dataResponse);
});

```

Ajoutez maintenant l'appel à notre API « WeatherApi » et l'affichage des résultats, juste après la récupération des coordonnées géographiques (latitudes et longitude).

Notez que les opérations sont encadrées par un « try-catch » (il se peut que l'API soit indisponible, par exemple...) :

```

if ("geolocation" in navigator) {
  /* geolocation is available */
  console.log('geolocation available');
  navigator.geolocation.getCurrentPosition(async position => {
    try {
      console.log(position.coords.latitude);
      lat = position.coords.latitude;
      document.getElementById('latitude').textContent = lat;
      console.log(position.coords.longitude);
      long = position.coords.longitude;
      document.getElementById('longitude').textContent = long;

      //Appel à l'API "WeatherApi"
      const apiUrl = `/weather/${lat}/${long}`; //on envoie 2 paramètres
      const response = await fetch(apiUrl);
      const json = await response.json();

      //Mise en forme des données reçues
      defra = json.current.air_quality["gb-defra-index"];
      city = json.location.name;
      country = json.location.country;
      condition = json.current.condition.text;
      temperature = json.current.temp_c;
      pm25 = json.current.air_quality.pm2_5;

      //Mise à jour des éléments à afficher sur la page en cours
      document.getElementById('city').textContent = city;
    } catch (error) {
      console.log('Erreur lors de la récupération des données');
    }
  });
}

```

```

        document.getElementById('country').textContent = country;
        document.getElementById('icon').src = json.current.condition.icon;
        document.getElementById('summary').textContent = condition;
        document.getElementById('temperature').textContent = temperature;
        document.getElementById('airQualityIndex').textContent = defra;
        document.getElementById('pm25').textContent = pm25;
        console.log(json);
    }
    catch(error){
        console.error(error);
    }
});
} else {
    /* geolocation IS NOT available */
    console.log('geolocation not available');
}

```

**Remarque :** la nouvelle route « /weather/ » que nous allons utiliser reçoit deux paramètres (latitude et longitude). Ce sont les coordonnées que l'API va utiliser pour nous renvoyer les données météorologiques les plus précises possible.

Il nous reste à créer notre nouvelle route « /weather » dans le fichier « index.js » :

```

//GET method route API WeatherApi
app.get('/weather/:lat/:long', async (request, response) => { //=> Quand c'est prêt, alors...
    console.log('I got a GET request for /weather');
    console.log(request.params);
    const lat = request.params.lat; //on récupère le 1er paramètre (la latitude)
    const long = request.params.long; //on récupère le 2ème paramètre (la longitude)

    try{
        const apiKey = "Votre ApiKey";
        const apiUrl =
            `http://api.weatherapi.com/v1/current.json?key=${apiKey}&q=${lat},${long}&aqi=yes`;

        const fetchResponse = await fetch(apiUrl);
        const json = await fetchResponse.json();
        response.json(json); //on renvoie la réponse au format JSON
        console.log(json);
    }
    catch(error){
        return response.json({
            status: 'GET Error'
        });
    }
});

```

Maintenant que nous avons codé l'appel à l'API qui permet d'afficher nos données, il nous reste à modifier notre fichier « index.js » pour enregistrer tous les éléments dans notre base de données SQLite lorsque l'on clique sur le bouton « Envoyer ».

### TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Modifier les méthodes « `app.post('/api', (request, response) »`, « `databaseInsert()` », « `Afficher()` », « `Inserer()` » et « `db.serialize()` » pour enregistrer toutes les données affichées dans la page « index.html ».

Revenons un instant sur la page « public\index.html ». L'index de la qualité de l'air (DEFRA) n'est pas très parlant...

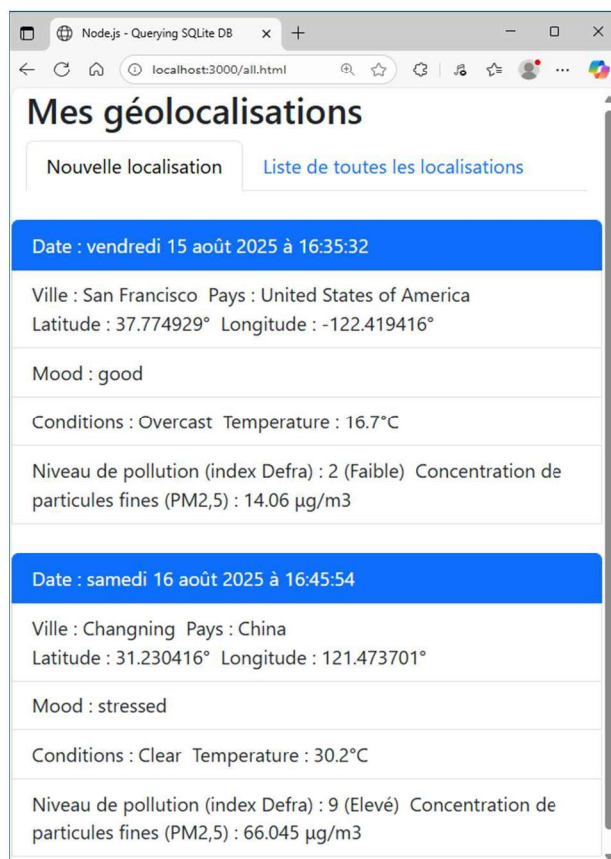
### TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Modifier l'affichage de l'index de qualité de l'air dans la fonction « `getCurrentPosition()` » pour afficher un résultat plus parlant (niveau de pollution faible, modéré, élevé, très élevé). Consultez la page suivante pour plus d'informations : <https://uk-air.defra.gov.uk/air-pollution/daq1>.

Notre application prend forme mais il manque encore une dernière chose : afficher l'ensemble des données enregistrées lorsqu'on ouvre la page « all.html ».

### TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Modifier la fonction « `GetData ()` » de la page « all.html » pour afficher toutes les données enregistrées dans la base.



## 8. Cacher une « API Key » avec les variables d'environnement

Dans notre projet, la clé qui permet d'accéder à l'API « weatherapi.com » est clairement visible dans notre code. Nous allons utiliser une variable d'environnement stockée dans un fichier « .env » pour cacher notre clé.

Commencez par Installer la bibliothèque « dotenv » en utilisant la commande suivante :  
« npm install dotenv ».

### TRAVAIL A FAIRE POUR LES ETUDIANTS :

- Etudiez la documentation sur la page <https://www.npmjs.com/package/dotenv> et cachez votre API key dans un fichier « .env » situé à la racine de votre projet.
- Modifiez le fichier « index.js » pour utiliser la variable d'environnement.
- Enregistrez votre projet dans un repository Github sans prendre en compte le fichier « .env » et les bibliothèques installées.